WQF7006 - COMPUTER VISION AND IMAGE PROCESSING

SEMESTER 1 2025/2026

MALAYSIAN SIGN LANGUAGE TRANSLATION CASE STUDY

PROJECT TEAM

OCCURENCE 3 GROUP 4

GROUP MEMBERS

Hin Jian Heng	**25069335**
In Jie Yang	**25065486**
Wang Ruoyu	**24082035**
Lee Wei Quan	**25068956**
Tee Chee Hong	**25069322**

SUPERVISOR

DR. ZAINAB MALIK

Table of Contents

ABSTRACT

The digital divide facing the deaf community in Malaysia highlights a critical lack of computational resources for Malaysian Sign Language (MSL). While Artificial Intelligence has successfully bridged communication gaps for major spoken languages, MSL remains a low-resource language in the computational domain. This group assignment presents the development of "MySign," an end-to-end AI-based translation system designed to facilitate inclusive communication.

Instead of using traditional static image classification, this study adopts a 3D landmark-based sequential approach. We utilized MediaPipe to extract skeletal keypoints from video data which reduces computational overhead while protecting user privacy. To determine the optimal architecture for real-world MSL recognition, the team simulated a professional R&D unit to train and evaluate four distinct deep learning architectures: Long Short-Term Memory (LSTM), 1D Convolutional Neural Networks (1D-CNN), Transformers, and Temporal Convolutional Networks (TCN).

Our experiments revealed that the TCN model was the superior performer, achieving the highest test accuracy among the candidates. It outperformed the complex Transformer and the localized 1D-CNN. This report details our full development lifecycle, including a data engineering pipeline that filtered the dataset to 52 distinct glosses and utilized data augmentation. Ultimately, we deployed the model via a client-side web application using ONNX, ensuring low-latency inference to practically meet Sustainable Development Goal (SDG) 10: Reduced Inequalities.

Keywords — Malaysian Sign Language (MSL), MediaPipe 3D Landmarks, TCN, Deep Learning, Client-side Inference.

1. INTRODUCTION

1.1 Background and Motivation

Sign language is the primary mode of communication for millions of deaf and hard-of-hearing individuals worldwide. It is a complex visual language that combines hand shapes, facial expressions, orientation, and body movement to convey meaning. However, unlike spoken languages which have translation tools like Google Translate or Siri, sign languages have lagged behind in technological development. This is particularly true for Malaysian Sign Language (MSL), or Bahasa Isyarat Malaysia.

MSL is distinct from American Sign Language (ASL) or British Sign Language (BSL), possessing its own grammar and vocabulary. Progress on translation tools has stalled because there just isn't enough labeled data for Malaysian Sign Language. This creates a significant barrier to daily interactions for the Malaysian deaf community, often requiring the presence of human interpreters for basic tasks such as visiting a clinic, ordering food, or attending a lecture.

The motivation behind this project is to break these barriers using Computer Vision. By teaching a machine to see and interpret MSL gestures, we aim to create a digital bridge that empowers Deaf individuals with greater independence.

1.2 Problem Statement

Developing an effective MSL translation system presents unique technical challenges.

1. **Visual Complexity:** MSL is not static. A single frame cannot distinguish between dynamic signs like "Pergi" (Go) and "Datang" (Come). The system must analyze the sequence of movements over time.
2. **Inter-class Similarity:** Many signs share the same hand shape but differ only in their position relative to the body (e.g., the sign for "Mother" vs. "Father"). A simple image classifier might fail to distinguish these without robust feature extraction.

3. **Real-time Constraint:** For a translation tool to be socially useful, it must operate in real-time. A delay of even two seconds breaks the flow of conversation, rendering the technology impractical.

1.3 Project Objectives

The primary objective of this project is to simulate a multidisciplinary AI development team to design, train, and deploy a prototype MSL recognition system. Specifically, we aim to:

- **Evaluate Sequential Architectures:** Compare the performance of LSTM, CNN, TCN, and Transformer models on a custom MSL dataset.
- **Design for Inclusion:** Create a user interface that prioritizes accessibility, ensuring the tool is usable by both deaf and hearing users.
- **Analyze Societal Impact:** Evaluate the solution through the lens of Sustainable Development Goals (SDGs) to ensure ethical compliance.

1.4 Scope

The scope of this case study is limited to the recognition of isolated sign language. We focus on classifying 52 discrete glosses (words/phrases) from video frames. While continuous sign language recognition (sentence-level) is the ultimate goal of the field, this project focuses on the fundamental building blocks: accurate word-level recognition in a controlled environment.

2. TEAM ROLES AND AGILE METHODOLOGY

To ensure the successful delivery of a prototype within the semester timeline, we adopted a simulated professional tech team structure. We utilized Agile methodology, conducting weekly sprint planning to align the diverse technical streams.

- **The AI Architect: In Jie Yang**

  Jie Yang served as the technical lead for the model development. His scope was strictly defined within the Python and Kaggle environments. He managed the data preprocessing pipeline (MediaPipe extraction) and engineered the four distinct models (LSTM, CNN, TCN, Transformer). He performed the hyperparameter tuning to identify the TCN as the optimal model.

- **The Frontend Engineer: Hin Jian Heng**

  Jian Heng was responsible for the functional implementation of the application. He built the client-side logic and integrate the trained model structure into a usable interface. He ensured the application logic could handle the sequence of landmarks input and display the predicted glosses with low latency.

- **The Frontend Designer: Wang Ruoyu**

  Ruoyu led the UI/UX design for the MySign web application. He produced a high-fidelity Canva prototype and defined the end-to-end user flow. The design covers key interaction states including camera on/off, camera permission denied guidance, dark/light mode, adjustable text size, and the translation interface layout.

- **The Product Manager: Lee Wei Quan**

  Wei Quan maintained the project timeline. He developed the Gantt chart and ensured the team met critical milestones (Data Collection, Model Training, Report). He also defined the specific use case (Service Counter Scenario) to guide the practical design of the app.

- **The Integration Lead: Tee Chee Hong**

  Chee Hong served as the Quality Assurance (QA) lead and reporter. He led the compilation of this report, synthesizing technical notes into academic writing. He also gave ideas on accessibility, ethics, and societal impact, ensuring the project aligned with SDG principles.

3. DATA COLLECTION

The foundation of any robust computer vision model is high-quality data. In the domain of deep learning, the model is only as good as the data it is fed. This section highlights the methodology used by the team to transform raw video footage into a balanced dataset suitable for training sequential neural networks.

3.1 Data Acquisition Strategy

Figure 3.1: Data collection setup utilizing a green screen environment to minimize background noise.

The dataset was constructed through a series of coordinated academic sessions (OCC 1, OCC 2, and OCC 3). Unlike public datasets which are often curated in studios, our dataset reflects a more realistic distribution.

To mitigate background noise and ensure the focus remained on the signer, we established a recording station within a classroom environment using a green cloth as the background to remove noise. While our eventual use of MediaPipe landmarks makes the model robust to background variations, this controlled environment during the data collection phase was crucial. It ensured that the initial video quality was high and that there were no distracting elements that might cause the landmark detection algorithms to fail during the pre-processing stage.

We recorded 90 distinct glosses, covering a range of vocabulary including greetings ("Assalamualaikum", "Hi"), nouns ("Ayah", "Emak"), verbs ("Makan", "Lari"), and adjectives ("Marah", "Suka").

3.2 Data Cleaning and Filtering

Upon a detailed inspection of the collected dataset, we identified several anomalies common in collaborative data collection that, if left unaddressed, would have introduced noise into the training process. Our audit revealed instances of misclassification where distinct variations of signs were mixed into incorrect parent folders. To ensure distinct class boundaries, we performed the following manual corrections:

1. **Gender correction:**

   Videos of 'anak_perempuan' (Daughter) found within the 'anak_lelaki′ (Son) folder were migrated back to their correct directory.

2. **Variation separation:**

   Samples of 'bapa_saudara' (Uncle) were extracted from the 'bapa' (Father) folder. Similarly, 'beli_2' and 'tolong_2', which represent specific variations of "Buy" and "Help", were separated from the root 'beli' and 'tolong' folders and placed into their own distinct classes.

Furthermore, we addressed class redundancy and nomenclature. The folders 'baik' and 'baik_2' were identified as representing the exact same concept and were consolidated into a single ground-truth class. Additionally, the remaining samples in the 'bapa' folder were reclassified and moved to 'ayah' to standardize the vocabulary list used for training.

Finally, we addressed the distribution imbalance. While popular signs like 'Hi' contained approximately 400 video samples, less common signs had significantly fewer. To ensure statistical significance, we established a stricter threshold. Any class containing fewer than 50 video samples was deemed

insufficient for generalization and was dropped from the dataset. This rigorous filtering process reduced our total class count from the original 90 down to a robust 52 classes.

3.3 Feature Extraction via MediaPipe Landmarks

Figure 3.2: MediaPipe holistic landmark extraction showing the skeletal wireframe used for training.

Feeding raw video frames directly into a neural network is computationally expensive and prone to overfitting on irrelevant features like the signer's shirt colour or skin tone. To address this, we adopted a landmark-based approach using Google MediaPipe. Instead of 2D representations, we extracted 3D coordinates for the hands and pose. The inclusion of the depth (z) axis proved important for identifying MSL signs that involve movements towards or away from the body, a nuance often lost in 2D projections.

This approach offers two distinct advantages. First, it provides massive dimensionality reduction. A standard video frame might contain over 150,000 pixel values, but converting this to a set of coordinates reduces the input to a mere few hundred floating-point numbers per frame. This allows for faster training and enables the final model to run in real-time on mobile devices. Second, it preserves privacy. The model never sees the user's face, only a mathematical wireframe of their movement.

3.4 Handling Imbalance: Sampling and Augmentation

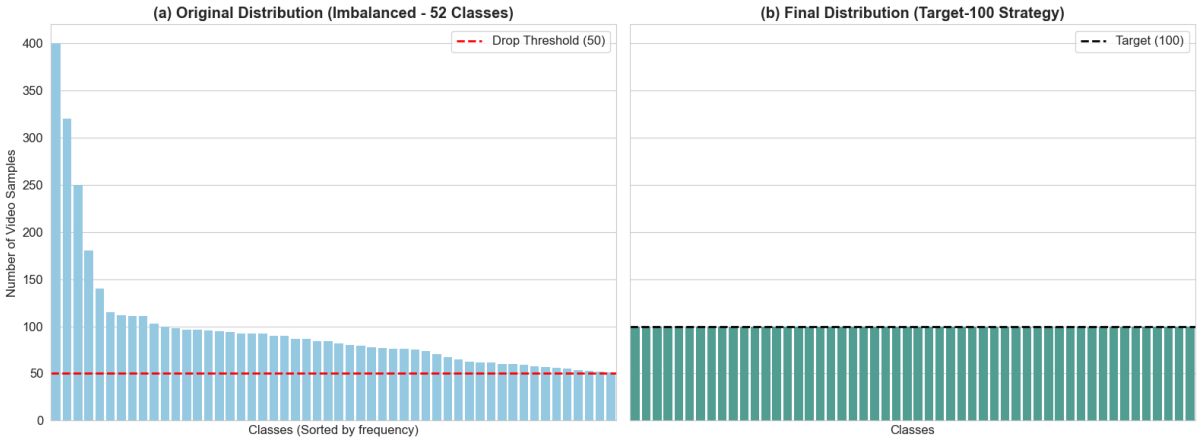

Figure 3.3: Class distribution before and after applying downsampling and augmentation strategies.

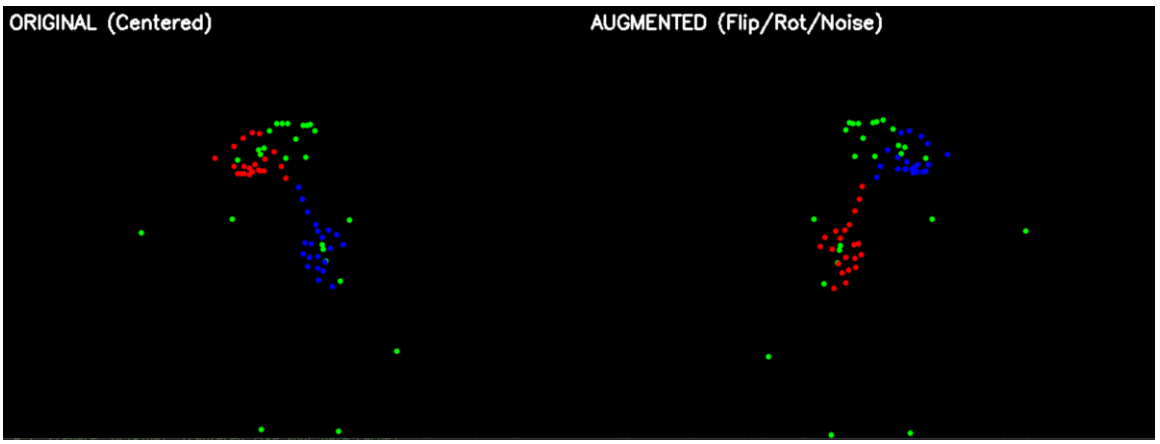

Figure 3.4: Visual comparison of a landmark skeleton before and after augmentation (Original vs. Jittered/Rotated)

Even after filtering, the dataset remained imbalanced (e.g., 400 samples for 'Hi' vs. 30 for 'Buang'). To prevent the model from becoming biased toward the majority classes, we ensured that every single class had exactly 100 samples for training.

1. **Downsampling:**

   For classes with more than 100 samples, we randomly selected 100 distinct samples and discarded the excess. This prevents the model from overfitting to the most frequent signs.

2. **Upsampling via Augmentation with Custom Dataset:**

   For classes with fewer than 100 samples, instead of generating static synthetic files to reach the 100-sample target, we implemented a dynamic approach using a custom torch.utils.data.Dataset class.

- **Dynamic Transformation:**

  Inside the training loop (specifically the __getitem__ method), we applied random augmentations to these samples in real-time. This includes mathematical jittering and random rotation. This technique ensures that the model sees a slightly different variation of the sample in every epoch, preventing overfitting more effectively than static augmentation.

- **Center Normalization:**

  Raw coordinates vary based on where the user stands in the frame. To make the model invariant to position, we implemented a normalization technique based on the shoulders. For every frame, we calculated the midpoint between the left and right shoulders and subtracted this value from all other landmarks. This mathematically centers the skeleton and ensures the model learns the relative motion of the hands around the body rather than absolute screen coordinates.

3.5 Data Splitting Strategy

To ensure a fair evaluation, we avoided simple random splitting and utilized a Stratified Group K-Fold approach. It is important that the same person does not appear in both the training and the testing set. Otherwise, the model might just recognize the person's body shape rather than the sign. Our split ensured that specific signers were isolated to specific sets.

Furthermore, we implemented strict sample grouping to prevent data leakage at the instance level. As seen in **Figure 3.5**, each student recorded the same gesture three times consecutively. We ensured that these triplet variations were treated as an indivisible unit. If the first repetition (e.g., _01) was assigned to the training set, the remaining two repetitions (_02 and _03) were automatically assigned to the same set. This guarantees that the model is tested on truly unseen data rather than memorizing the specific environmental conditions or clothing of a specific recording session.

The cleaned data was first divided into 85% Training and 15% Testing. From the training set, we further set aside 15% as a Validation set to tune hyperparameters and trigger early stopping.

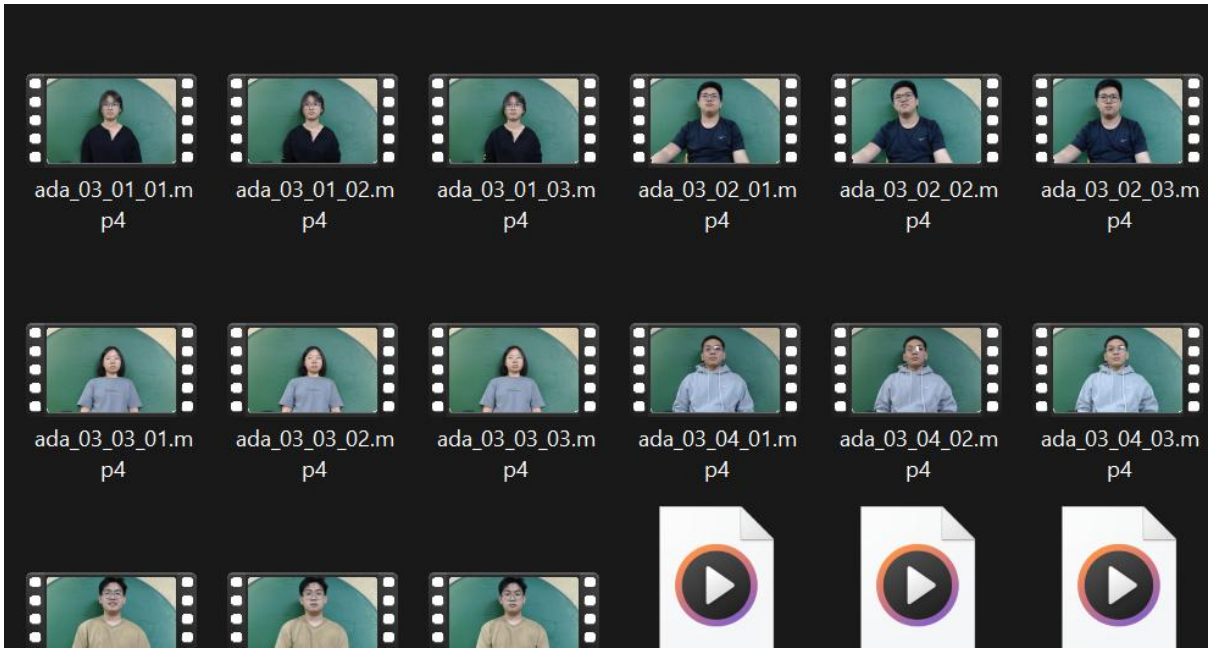

Figure 3.5: Recorded videos for the gesture ‘ada’. Each student will perform the same gesture 3 times and stored as 3 separate videos.

4. MODEL TRAINING

With a clean, balanced, and 3D landmark-based dataset, we proceeded to the modeling phase. Rather than relying on a single architecture, we conducted a comparative study of four deep learning paradigms to identify the best model for MSL translation.

4.1 Comparative Architectures

We evaluated four distinct architectures, each chosen for its theoretical advantages in handling time-series data. First, we tested a **Custom LSTM (Long Short-Term Memory)** network. It was once the standard for sequence tasks, so we wanted to test if LSTMs could effectively capture the grammar of hand movements over time. Second, we implemented a **SignLanguageTransformer**. This model uses self-attention mechanisms to weigh the importance of different frames. While state-of-the-art in NLP, we aimed to test if its complexity was justified for skeletal data.

Third, we tested a **SignLanguageCNN (1D-CNN)**. This architecture treats the time series of coordinates like a 1D image, sliding kernels across the time axis to detect local patterns. Finally, we evaluated a **SignLanguageTCN (Temporal Convolutional Network)**. TCN utilizes dilated convolutions to process sequences. Unlike the CNN which focuses on local features, the TCN's dilated layers allow it to expand its receptive field exponentially, enabling it to see the entire history of a gesture simultaneously without the massive computational overhead of a Transformer or the forgetting issues of an LSTM.

4.2 Experimental Setup

The training was conducted in the Kaggle environment, using the NVIDIA Tesla P100 GPU to accelerate the tensor operations. The codebase was built using PyTorch. To facilitate reproducibility, the following key libraries were integrated into our workflow (as referenced in the project code):

- torch and torch.nn: For defining the neural network layers and loss functions.
- torch.optim: For the optimization algorithms.

- sklearn.model_selection.StratifiedGroupKFold: For the robust data splitting described in Section 3.5.
- mediapipe: For the landmark extraction pipeline.
- opencv-python (cv2): For video frame handling.
- numpy: For high-performance matrix manipulations of the coordinate data.

4.3 Hyperparameter Tuning & Optimization

We applied a standardized training protocol across all four models to ensure a fair comparison:

1. **Optimizer Selection:**

   We opted for the AdamW optimizer (Adam with Weight Decay) instead of the standard SGD (Stochastic Gradient Descent). AdamW creates an adaptive learning rate for each parameter. Since some MSL signs are very distinct and learned quickly, while others are subtle and learned slowly, AdamW handled this better than SGD.

2. **Learning Rate Scheduling:**

   We implemented a ReduceLROnPlateau scheduler with a factor of 0.1. This monitors the Validation Loss. If the model stops improving for a set number of epochs, the learning rate is then reduced and allows the model to settle into a more precise local minimum.

3. **Early Stopping:**

   To prevent overfitting, we monitored the validation loss. If the loss did not improve for 20 consecutive epochs, the training was automatically halted and the best-performing weights were restored.

4.4 Client-Side Model Export

Upon achieving optimal convergence, we did not rely solely on the standard PyTorch (.pth) weights. Instead, we explicitly converted and exported the final trained TCN model into the ONNX (Open Neural Network Exchange) format.

Justification for ONNX:

This architectural decision was driven entirely by the needs of the frontend application.

1. **Interoperability:** ONNX acts as a universal bridge, allowing our model trained in a Python/PyTorch environment to be seamlessly loaded into the client-side environment (JavaScript or Swift/Kotlin) without requiring the heavy PyTorch library to be installed on the user's device.
2. **Inference Speed:** The ONNX Runtime is highly optimized for inference. By converting the model, we unlocked graph optimizations (like node fusion) that significantly reduce the computational cost during the forward pass. This ensures that the mobile app remains responsive and does not drain the user's battery which is a critical requirement for a real-time accessibility tool.

5. FINDINGS

5.1 Quantitative Model Results

After training all four models on the balanced dataset, we evaluated them on the held-out test set. The results provided a clear hierarchy of performance:

Table 5.1: Comparative Performance Metrics

Model Architecture	Test Accuracy
Custom LSTM	77.57%
1D-CNN	93.00%
Transformer	85.86%
TCN (Best)	**94.14%**

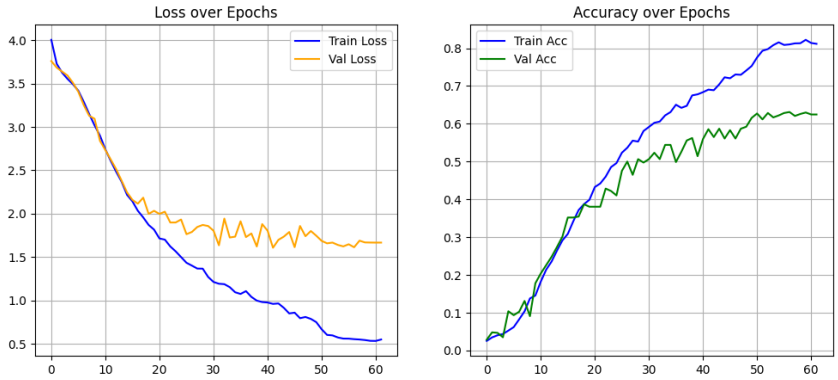

Figure 5.1: (a) Training and validation loss over epochs graph of LSTM (b) Training and validation accuracies over epochs graph of LSTM

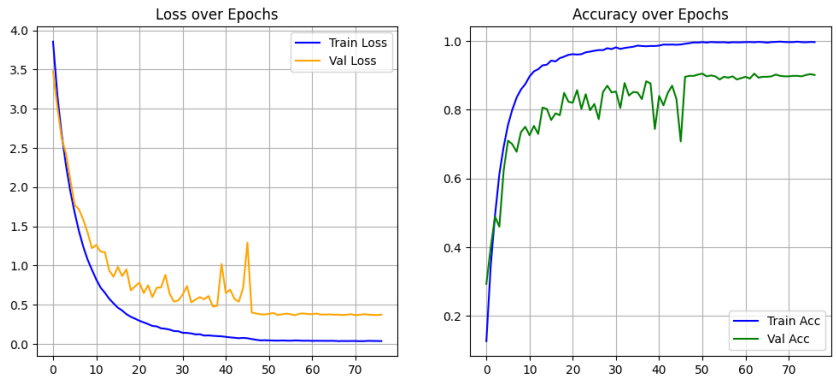

Figure 5.2: (a) Training and validation loss over epochs graph of CNN (b) Training and validation accuracies over epochs graph of CNN

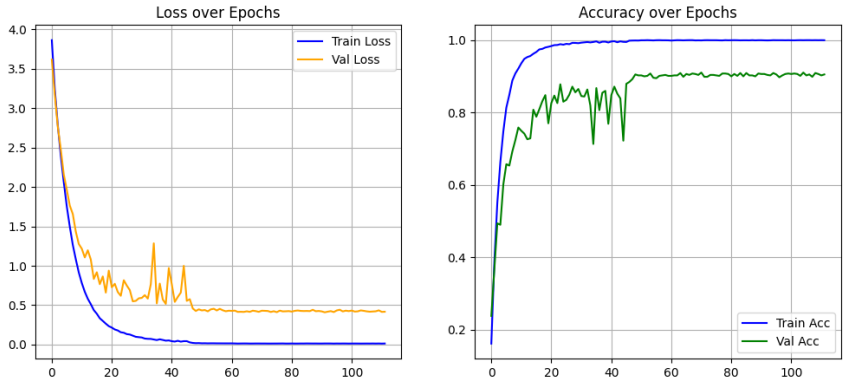

Figure 5.3: (a) Training and validation loss over epochs graph of TCN (b) Training and validation accuracies over epochs graph of TCN

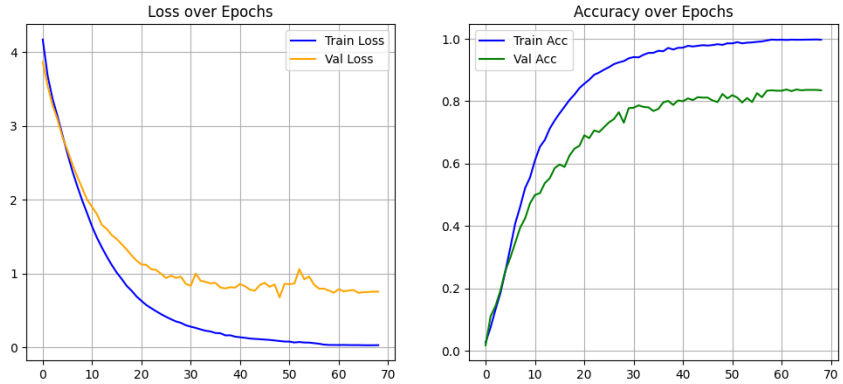

Figure 5.4: (a) Training and validation loss over epochs graph of Transformer (b) Training and validation accuracies over epochs graph of Transformer

The Temporal Convolutional Network (TCN) emerged as the superior architecture, achieving a test accuracy of 94.14%. It was closely followed by the 1D-CNN at 93.00%. The Transformer performed respectably at 85.86%, while the LSTM lagged significantly behind at 77.57%.

Regarding reliability, the TCN demonstrated an exceptionally high confidence level. In the context of the output layer, the model rarely hesitated. For correct predictions, it typically assigned a probability score exceeding 0.95 to the correct class, indicating it had learned robust and distinguishable temporal features for the signs.

5.2 Critical Analysis: Why TCN Outperformed Others?

The superiority of the TCN over the Transformer and LSTM can be attributed to the nature of sign language data. Unlike natural language sentences which have complex and long-range grammatical dependencies, isolated sign glosses are defined by specific and high-velocity kinematic strokes (a start, a movement, and a stop).

The TCN with its dilated convolutions, may have strike the perfect balance. It captures the local motion details like a CNN while simultaneously maintaining a global view of the entire gesture's duration. The Transformer, while powerful, is data-hungry and likely underfitted given our dataset size of 100 samples per class. The LSTM, meanwhile, struggled with the vanishing gradient problem over the longer sequences generated by our 30-frame sampling.

5.3 Error Analysis

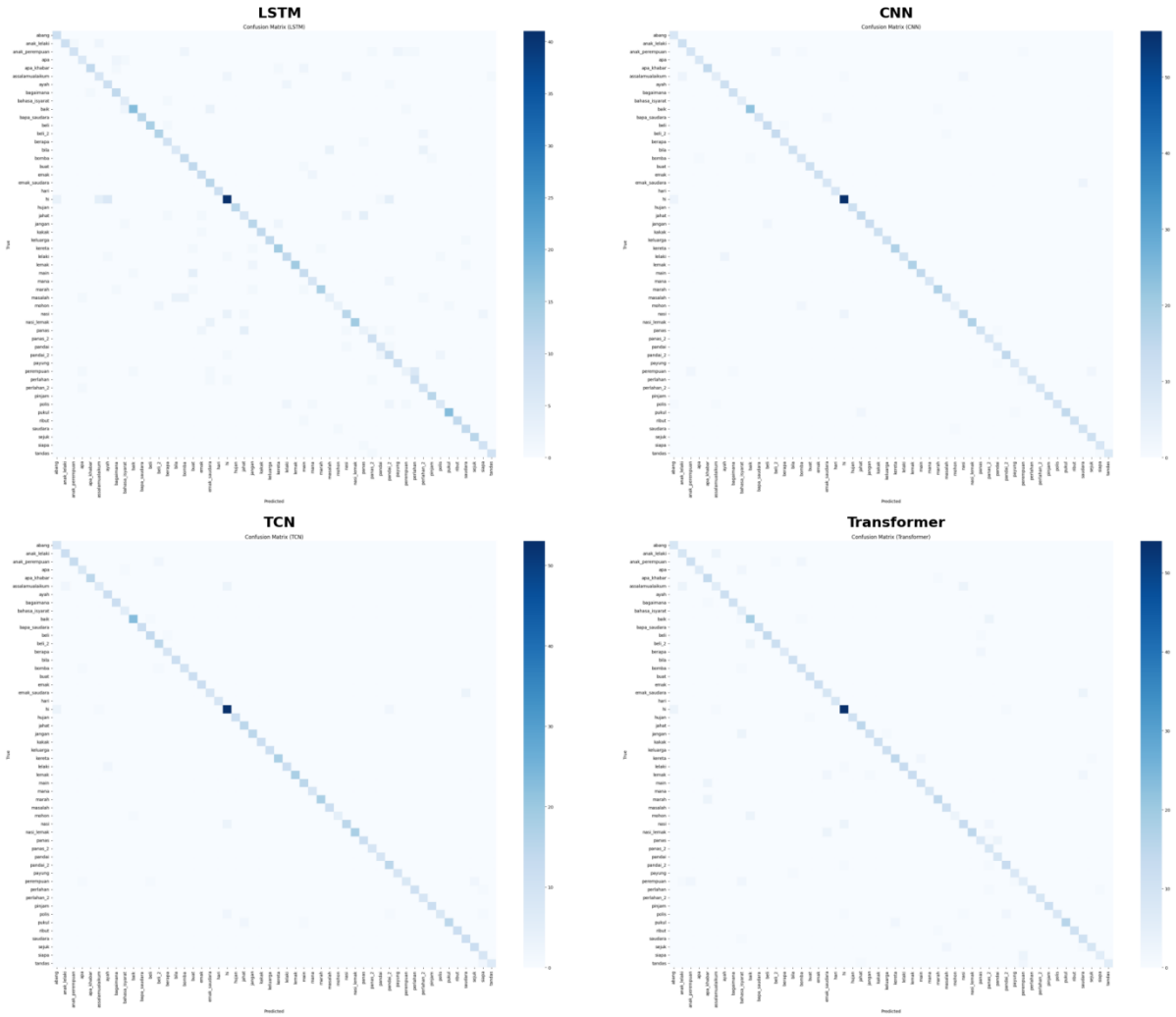

Figure 5.5: (a) Confusion Matrix of LSTM **(b)** Confusion Matrix of CNN
(c) Confusion Matrix of TCN **(d)** Confusion Matrix of Transformer

To understand the limitations of our best model, we have generated the confusion matrices of these models. The application consistently struggled with three specific glosses: "Assalamualaikum", "Bahasa Isyarat", and "Emak Saudara".

- **Inter-Class Similarity:** The primary source of error was between signs that share identical hand shapes but differ subtly in depth or position. For example, the signs for "Bapa" (Father) and "Bapa Saudara" (Uncle) both utilize a similar hand configuration positioned near the cheek area. Since MediaPipe's z-coordinate (depth) can be noisy, the CNN occasionally confused these two.

- **Motion Blur:** Fast-moving dynamic signs like "Lari" (Run) showed slightly lower recall scores. This is likely because rapid motion causes smearing in the video frames, leading to less accurate landmark detection by MediaPipe in those specific frames.

5.4 UI/UX Design Prototype (Canva)

Before implementation, we created a high-fidelity UI/UX prototype for the “MySign” application to define the interface structure and end-to-end user flow. The prototype specifies the layout of three key panels—Video Input, Translation Pad, and Gesture Reference—and highlights accessibility-first design choices, including dark/light mode and adjustable text size options. As illustrated in Figure 5.6, the initial screen provides clear onboarding guidance for camera-based recognition and presents the primary controls (Start/Stop/Reset/Watch Demo) in a simple, high-contrast layout. During operation, the interface is designed to surface real-time feedback, including camera status and frame rate, while presenting the predicted gloss with confidence and maintaining a readable prediction history. Figure 5.7 demonstrates the light-mode recognition state with landmark visualization and the translation workflow (current prediction, history, and TTS/Speak). For users who prefer reduced glare, Figure 5.8 indicates the corresponding dark-mode recognition state, preserving the same information hierarchy and interaction patterns for consistent usability.

UI/UX prototype (Canva): https://www.canva.com/design/DAG-NkxPyC0/NQjiFAuV8mfquobGEgbx7g/edit?utm_content=DAG-NkxPyC0&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

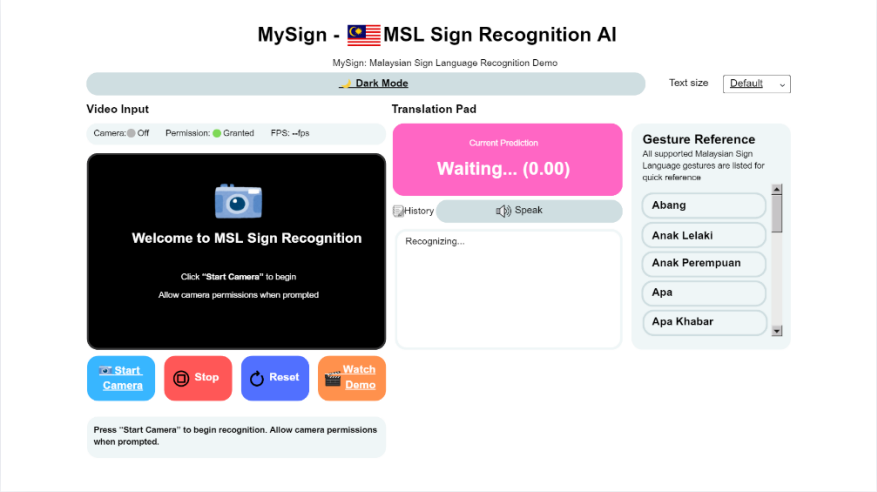

Figure 5.6: MySign UI initial screen (idle state), showing the overall layout and accessibility controls (dark mode and text size).

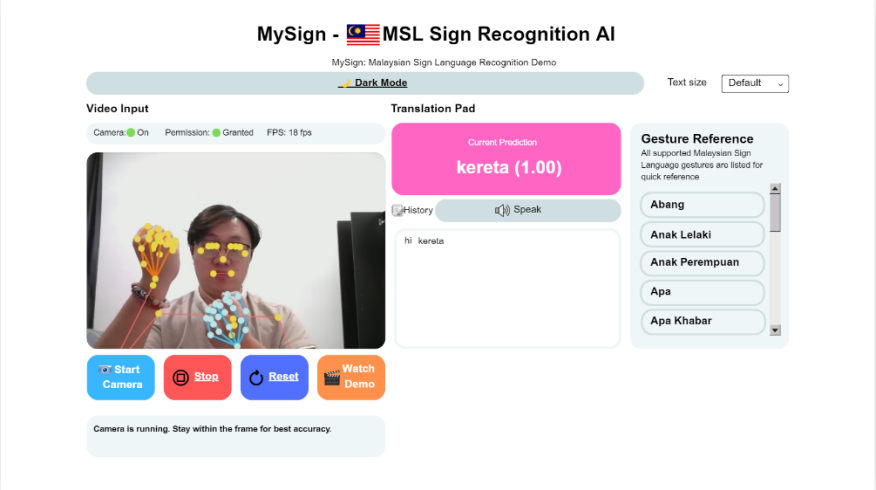

Figure 5.7: MySign UI in light mode during recognition, displaying landmark overlay, current prediction, history, and TTS (Speak).

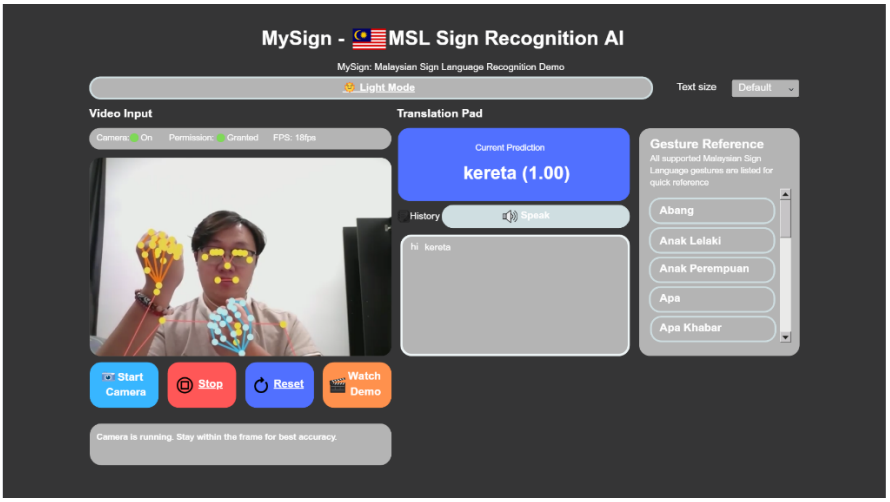

Figure 5.8: MySign UI in dark mode during recognition, preserving the same layout while improving readability in low-light conditions.

5.5 Application Mock-up: "MySign"

To demonstrate the practical utility of our model, the frontend team built a fully functional mobile and web-based prototype hosted on Cloudflare called "MySign." The architecture is distinctively serverless. “MySign” performs all inference on the client side. By utilizing ort.min.js (ONNX Runtime Web), the browser downloads the TCN model once and runs it using the device's own CPU or GPU. This results in minimal latency. There is no network round-trip time for video data, making the translation appear instantaneous. This architecture also ensures that the application is scalable and cost-effective, as there are no expensive backend GPU servers to maintain.

The core functionality provides real-time translation. Users point their camera at a signer and the app overlays the predicted text. We implemented several user-centric features. These included light and dark mode to reduce eye strain, adjustable text size for elderly users or those with visual impairments, and a text-to-speech (TTS) feature. The TTS allows the app to vocalize the translated sign. This enables a two-way conversation where a Deaf person signs, the app speaks for them, and the hearing person can listen.

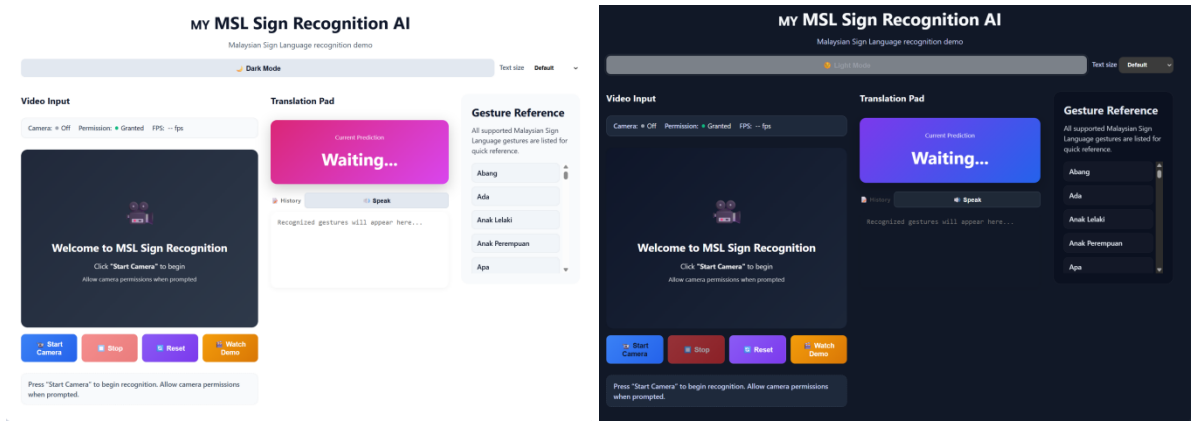

Figure 5.9: The landing page of the mock-up app “MySign”. Figure on the left shows the light mode while the other figure shows the dark mode.

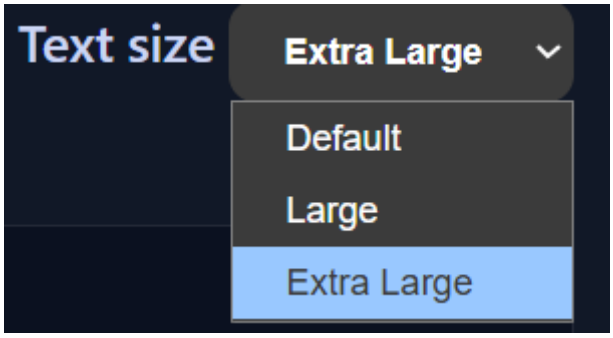

Figure 5.10: Accessibility option for users to select larger text sizes.

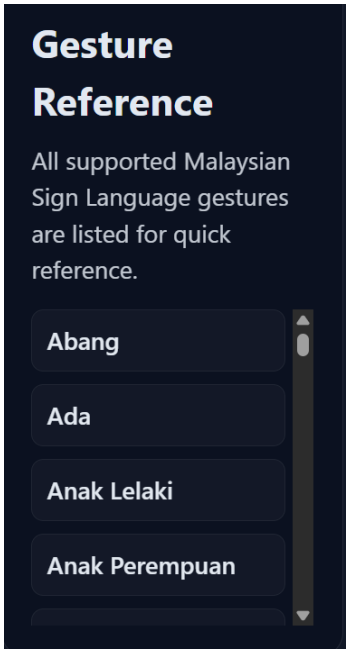

Figure 5.11: A list of recorded gesture references where users could see which possible movements can be translated.

6. DISCUSSION: ETHICS AND SOCIETAL IMPACT

6.1 Alignment with Sustainable Development Goals (SDGs)

This project is a direct technological intervention supporting the UN SDGs:

- **SDG 10 (Reduced Inequalities):** By providing a free translation tool, we empower the deaf community to access services (banking, healthcare, education) without strictly relying on human interpreters who are often scarce and expensive.

- **SDG 4 (Quality Education):** "MySign" can be used as a learning tool for hearing individuals to learn MSL vocabulary through real-time feedback.

6.2 Accessibility and Inclusion

Technology for the deaf must be built with a strong ethical foundation. In our simulated role as ethics officers, we evaluated the inclusivity of our architecture. Our decision to use MediaPipe 3D Landmarks mitigates racial and skin-tone bias. Because the model sees a mathematical skeletal wireframe rather than raw pixel data, it couldn't detect the user's skin colour or background lighting conditions.

However, ethical inclusion extends beyond code. We designed the user interface to be transparent about the model's confidence levels. By showing the prediction probability, we ensure users understand that the AI is just a probabilistic tool.

6.3 Data Privacy

Privacy is a major concern when cameras are involved. Our architecture addresses this through edge computing. Because our chosen TCN model is lightweight and efficient, it does not require a massive cloud server to run. The model can be deployed directly on the user's smartphone via the ONNX runtime. This means that no video data ever leaves the user's device. The camera feed is processed locally, protecting the user's privacy and ensuring that sensitive conversations remain confidential.

7. CONCLUSION

This project successfully simulated the end-to-end lifecycle of an AI product, moving from messy data to a polished and functional prototype. We demonstrated that a Temporal Convolutional Network (TCN) which is trained in 3D MediaPipe Landmarks, is the optimal solution for this task, achieving a test accuracy of 94.14%.

Our comparative analysis highlighted that for specific motion-based tasks with limited data, specialized architectures like TCNs outperform generalist giants like Transformers. Furthermore, our successful deployment on Cloudflare shows that modern web browsers are capable of running complex deep learning models in real-time.

To elevate "MySign" from an academic prototype to a deployable product, future work should focus on two areas. First, we should explore hybrid input data. While landmarks are efficient, they miss facial expressions, which are grammatically important in MSL. Future models should fuse landmark data with cropped video data of the face. Second, we recommend implementing a continuous learning loop, where users can flag incorrect translations. This anonymized data would be used to retrain the model, allowing it to get smarter and adapt to regional accents over time.

8. DECLARATION OF AI TOOLS

In accordance with the Universiti Malaya academic integrity policy and the assignment guidelines, Group 4 declares the transparent use of the following AI tools during the development of this project:

1. **AI Studio (Gemini 3 Pro):**
    - Used by the Integration Lead to refine the academic tone of the report and ensure grammatical accuracy. It is also used to brainstorm the mathematical logic for the custom MSL Dataset and center normalization techniques. All AI-generated text and code suggestions were reviewed, tested, and modified by human team members. No content was copy-pasted blindly.
2. **GitHub Copilot:**
    - Used by the Frontend Engineer to accelerate the generation of the index.html boilerplate, the JavaScript logic for the ONNX runtime-web integration, and the UI styling.

The final content, including all analysis, metric evaluations, and conclusions, has been critically reviewed, verified, and edited by the group members to ensure accuracy and originality. No AI-generated output was included without human oversight and verification.

9. APPENDIX

The complete source code, dataset handling scripts, and model architecture definitions can be accessed through the following links:

- **GitHub Repository for Website:**
  https://github.com/JianHengHin0831/MSL_Streamlit_App

- **Model Training and Data Preprocessing Source Code:**
  https://drive.google.com/drive/folders/1e7yC9Njd8fv92oJLSaZGo17eBwrv2wjY?usp=sharing
- **Deployed Web Application:**
  https://msl-streamlit-app.pages.dev/